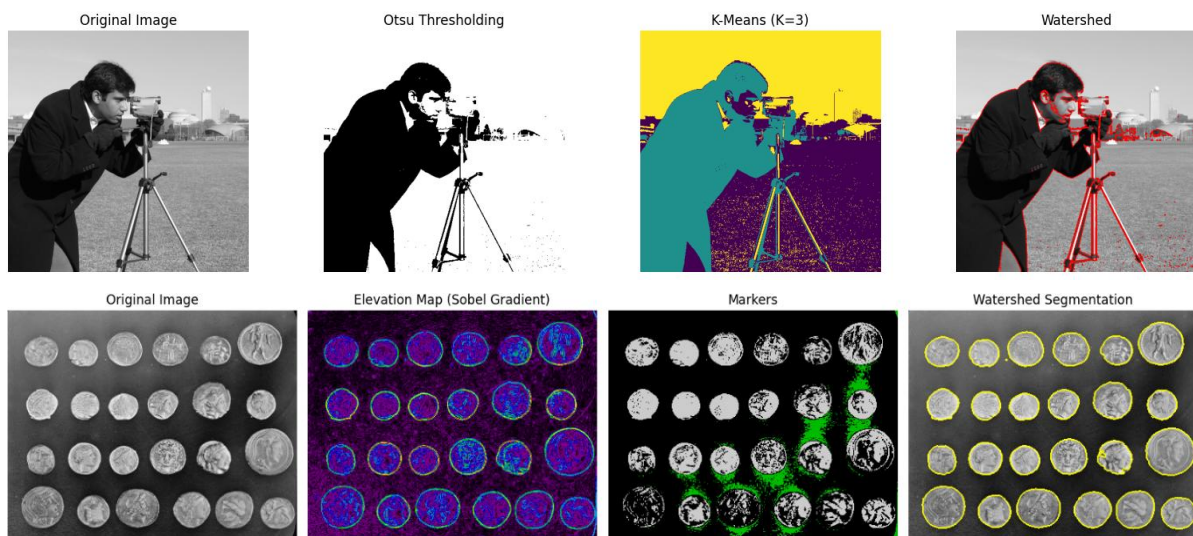




POLITEKNIK NEGERI SEMARANG
JURUSAN TEKNIK ELEKTRO
PROGRAM STUDI STR TEKNOLOGI REKAYASA KOMPUTER

JOBSHEET PRAKTIKUM
PENGOLAHAN CITRA
JOBSHEET 04: SEGMENTASI GAMBAR

Comparison of Segmentation Methods



Dosen Pengampu:
Ir. Prayitno, S.ST., M.T., Ph.D.

Nama Mahasiswa: _____

NIM: _____

Kelompok: _____



Semester Genap 2025

A. Tujuan Instruksional Khusus

Mahasiswa dapat menguraikan konsep segmentasi citra dengan indikator:

1. Menjelaskan konsep dasar dan tujuan segmentasi citra dalam pengolahan citra digital.
2. Menguraikan prinsip kerja metode segmentasi berbasis thresholding (ambang batas), termasuk thresholding global dan metode Otsu.
3. Menjelaskan konsep metode segmentasi berbasis region (wilayah), seperti region growing.
4. Memahami penerapan metode clustering (pengelompokan), seperti K-Means, untuk segmentasi citra.
5. Mengimplementasikan berbagai teknik segmentasi citra menggunakan Python dan pustaka scikit-image.
6. Menganalisis dan membandingkan hasil dari berbagai metode segmentasi pada citra yang berbeda.
7. Mengevaluasi kelebihan dan kekurangan masing-masing metode segmentasi.

B. Dasar Teori

1. Konsep Dasar Segmentasi Citra

Segmentasi citra adalah proses mempartisi atau membagi citra digital menjadi beberapa segmen (himpunan piksel) yang memiliki kesamaan atribut tertentu, seperti intensitas, warna, atau tekstur. Tujuan utama segmentasi adalah untuk menyederhanakan representasi citra menjadi sesuatu yang lebih bermakna dan lebih mudah dianalisis. Segmentasi merupakan langkah penting dan seringkali fundamental dalam banyak aplikasi visi komputer dan analisis citra, termasuk deteksi objek, analisis citra medis, pengenalan pola, dan pemrosesan citra satelit. Setiap piksel dalam suatu region dianggap homogen berdasarkan kriteria tertentu, dan region yang berbeda harus memiliki perbedaan signifikan dalam kriteria tersebut.

2. Metode Segmentasi Berbasis Thresholding (Ambang Batas)

Metode ini adalah salah satu teknik segmentasi klasik yang paling sederhana dan umum digunakan, terutama untuk citra dengan perbedaan kontras yang jelas antara objek dan latar belakang.

- **Thresholding Global:** Menentukan satu nilai ambang (threshold) T untuk seluruh citra. Piksel dengan intensitas di atas T diklasifikasikan ke dalam satu kelas (misalnya, objek), dan piksel dengan intensitas di bawah atau sama dengan T diklasifikasikan ke kelas lain (misalnya, latar belakang).
- **Metode Otsu:** Merupakan metode thresholding otomatis yang menentukan nilai ambang optimal dengan memaksimalkan varians antar kelas (between-class variance) atau meminimalkan varians intra-kelas (within-class variance). Metode ini efektif ketika histogram intensitas citra bersifat bimodal, namun seperti metode thresholding lainnya, dapat mengalami kesulitan pada kondisi pencahayaan yang bervariasi atau noise.

3. Metode Segmentasi Berbasis Region (Wilayah)

Metode ini bekerja dengan mengelompokkan piksel menjadi region berdasarkan kedekatan spasial dan kesamaan properti.

- **Region Growing:** Dimulai dengan satu atau lebih piksel "seed" (benih). Piksel-piksel tetangga dari seed kemudian diperiksa. Jika piksel tetangga memenuhi kriteria kesamaan (misalnya, perbedaan intensitas di bawah ambang batas tertentu), piksel tersebut ditambahkan ke region. Proses ini berlanjut hingga tidak ada lagi piksel tetangga yang memenuhi kriteria untuk ditambahkan ke region mana pun. Efektivitasnya dapat dipengaruhi oleh pemilihan seed dan kriteria kesamaan yang digunakan.

4. Metode Segmentasi Berbasis Clustering (Pengelompokan)

Metode ini mengelompokkan piksel berdasarkan fitur-fiturnya (misalnya, intensitas, warna) ke dalam sejumlah kluster (kelompok) tanpa memerlukan informasi spasial secara eksplisit pada tahap awal.

- **K-Means Clustering:** Merupakan algoritma clustering populer yang bertujuan mempartisi N piksel ke dalam K kluster yang berbeda dan non-overlapping. Setiap piksel dimasukkan ke dalam kluster dengan mean (pusat kluster) terdekat. Algoritma ini bersifat iteratif, memperbarui pusat kluster dan keanggotaan piksel hingga konvergen. Dalam konteks segmentasi, setiap kluster mewakili satu segmen dalam citra. Meskipun klasik, K-Means masih banyak digunakan dan dikembangkan, namun memiliki keterbatasan seperti sensitivitas terhadap pemilihan pusat awal dan penentuan nilai K .

5. Evaluasi Segmentasi

Mengevaluasi hasil segmentasi seringkali bersifat kualitatif (visual) dan kuantitatif. Metrik kuantitatif biasanya memerlukan citra *ground truth* (segmentasi ideal yang diketahui) untuk perbandingan, menggunakan metrik seperti Jaccard Index (Intersection over Union - IoU) atau Dice Coefficient.

C. Alat dan Bahan

Untuk dapat melaksanakan praktik jobsheet ini diperlukan:

- Komputer dengan Python terinstal
- Pustaka Python: scikit-image, matplotlib, numpy
- Contoh gambar dari scikit-image

D. Langkah Kerja Praktik

Praktikum 1. Segmentasi Menggunakan Thresholding Global dan Otsu

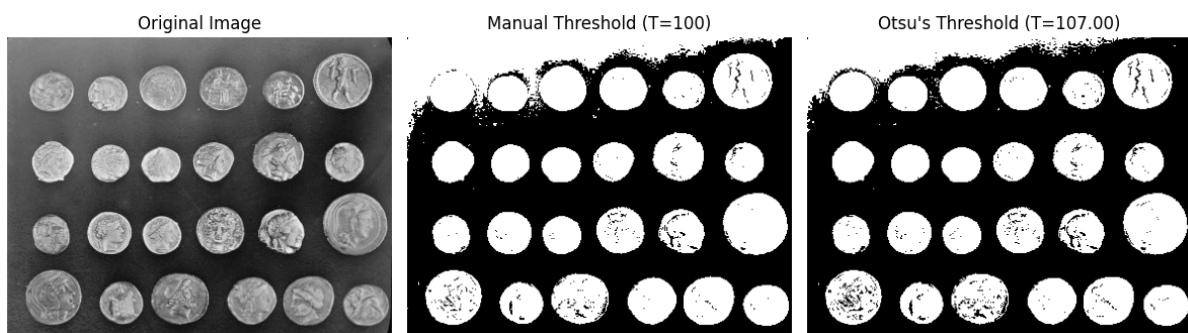
Praktikum ini bertujuan menerapkan metode segmentasi berbasis thresholding pada citra grayscale. Mahasiswa akan membandingkan hasil thresholding dengan nilai ambang manual (global) dan nilai ambang otomatis yang ditentukan oleh metode Otsu.

Kode Praktikum:

```
01: import matplotlib.pyplot as plt
02: from skimage import data, filters, img_as_ubyte
03: from skimage.color import rgb2gray
04:
05: # 1. Memuat citra (contoh: coins)
06: image_coins = data.coins() # Citra sudah grayscale
```

```
07:
08: # 2. Thresholding Global (manual)
09: # Tentukan nilai ambang manual, misal 100
10: thresh_manual = 100
11: binary_manual = image_coins > thresh_manual
12:
13: # 3. Thresholding Otsu
14: thresh_otsu = filters.threshold_otsu(image_coins)
15: binary_otsu = image_coins > thresh_otsu
16:
17: # 4. Visualisasi Hasil
18: fig, axes = plt.subplots(ncols=3, figsize=(12, 4))
19: ax = axes.ravel()
20:
21: ax[0].imshow(image_coins, cmap=plt.cm.gray)
22: ax[0].set_title('Original Image')
23: ax[0].axis('off')
24:
25: ax[1].imshow(binary_manual, cmap=plt.cm.gray)
26: ax[1].set_title(f'Manual Threshold (T={thresh_manual})')
27: ax[1].axis('off')
28:
29: ax[2].imshow(binary_otsu, cmap=plt.cm.gray)
30: ax[2].set_title(f'Otsu\'s Threshold (T={thresh_otsu:.2f})')
31: ax[2].axis('off')
32:
33: plt.tight_layout()
34: plt.show()
35:
36: # Menampilkan nilai threshold Otsu
37: print(f"Nilai threshold Otsu yang ditemukan: {thresh_otsu}")
```

Hasil Praktikum.



Amati perbedaan hasil segmentasi antara thresholding manual dan Otsu. Apakah metode Otsu berhasil memisahkan objek (koin) dari latar belakang dengan baik pada citra ini?

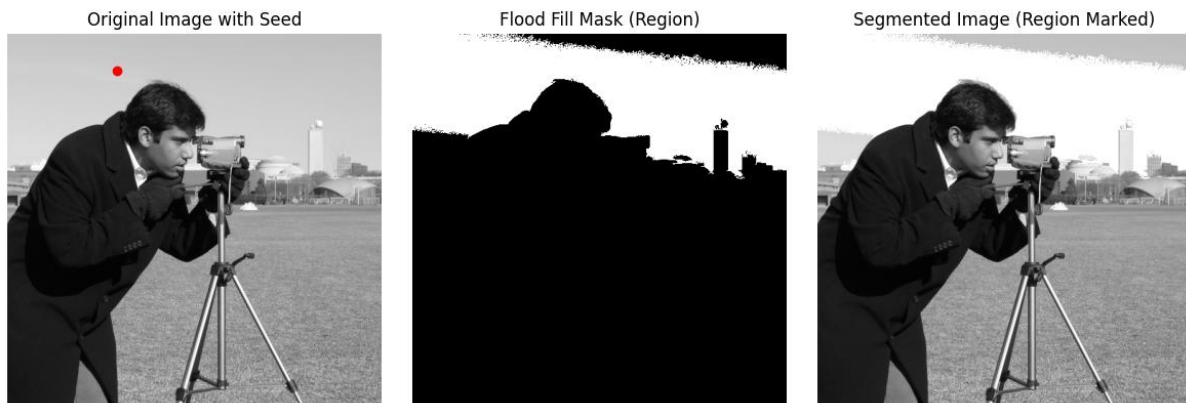
Praktikum 2. Segmentasi Menggunakan Region Growing (Contoh Sederhana)

Praktikum ini mendemonstrasikan konsep dasar region growing. Kita akan menggunakan fungsi `flood` dari `skimage.segmentation` sebagai contoh pendekatan berbasis konektivitas yang mirip dengan region growing.

Kode Praktikum:

```
01: import numpy as np
02: import matplotlib.pyplot as plt
03: from skimage import data, segmentation, color
04:
05: # 1. Memuat citra (contoh: camera)
06: image_camera = data.camera()
07:
08: # 2. Tentukan titik 'seed' (benih)
09: # Misal, kita pilih titik di area langit (misal, koordinat y=50, x=150)
10: seed_point = (50, 150)
11:
12: # 3. Terapkan algoritma flood fill (mirip region growing)
13: # 'tolerance' menentukan seberapa besar perbedaan intensitas yang
    diizinkan
14: flood_mask = segmentation.flood(image_camera, seed_point, tolerance=10)
15:
16: # 4. Buat citra tersegmentasi (tandai region yang 'tumbuh')
17: segmented_image = np.copy(image_camera)
18: segmented_image[flood_mask] = 255 # Tandai region dengan warna putih
19:
20: # 5. Visualisasi Hasil
21: fig, axes = plt.subplots(ncols=3, figsize=(12, 4))
22: ax = axes.ravel()
23:
24: ax[0].imshow(image_camera, cmap=plt.cm.gray)
25: ax[0].plot(seed_point[1], seed_point[0], 'ro') # Tandai seed point
26: ax[0].set_title('Original Image with Seed')
27: ax[0].axis('off')
28:
29: ax[1].imshow(flood_mask, cmap=plt.cm.gray)
30: ax[1].set_title('Flood Fill Mask (Region)')
31: ax[1].axis('off')
32:
33: ax[2].imshow(segmented_image, cmap=plt.cm.gray)
34: ax[2].set_title('Segmented Image (Region Marked)')
35: ax[2].axis('off')
36:
37: plt.tight_layout()
38: plt.show()
```

Hasil Praktikum



Amati region yang dihasilkan oleh algoritma `flood`. Bagaimana pengaruh parameter `tolerance` terhadap ukuran region yang 'tumbuh'? Cobalah mengubah titik seed dan nilai `tolerance`.

Praktikum 3. Segmentasi Citra Berwarna Menggunakan K-Means Clustering

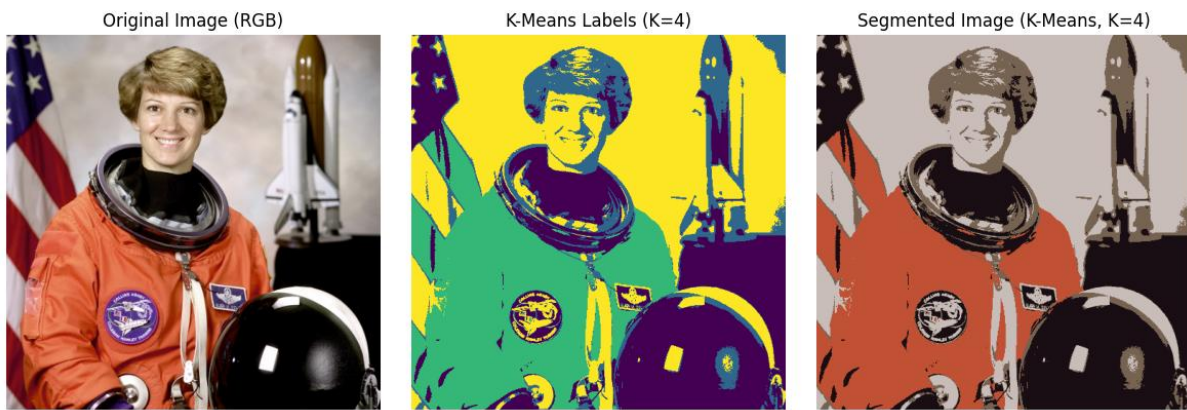
Praktikum ini menerapkan K-Means clustering untuk mensegmentasi citra berwarna berdasarkan nilai warna piksel.

Kode Praktikum:

```
01: import numpy as np
02: import matplotlib.pyplot as plt
03: from skimage import data, io
04: from sklearn.cluster import KMeans
05: from skimage.color import rgb2lab, lab2rgb
06: import warnings
07:
08: # 1. Memuat citra berwarna (contoh: astronaut)
09: image_astro = data.astronaut()
10: # Konversi ke float untuk perhitungan
11: image_astro_float = image_astro.astype(float) / 255.0
12:
13: # 2. Reshape citra menjadi array piksel [jumlah_piksel, jumlah_fitur]
14: # Fitur bisa RGB atau Lab. Ruang warna Lab seringkali lebih baik untuk
    persepsi warna.
15: # Konversi ke Lab
16: image_lab = rgb2lab(image_astro_float)
17: rows, cols, dims = image_lab.shape
18: pixel_features = image_lab.reshape(rows * cols, dims)
19:
20: # 3. Terapkan K-Means Clustering
21: # Tentukan jumlah klaster (segmen) yang diinginkan, misal K=4
22: n_clusters = 4
23: kmeans = KMeans(n_clusters=n_clusters, random_state=0, n_init=10) #
    n_init='auto' jika versi sklearn baru
```

```
24: # Hiding FutureWarnings related to default value of n_init
25: with warnings.catch_warnings():
26:     warnings.simplefilter("ignore")
27:     pixel_labels = kmeans.fit_predict(pixel_features)
28:
29: # 4. Reshape label kembali ke bentuk citra
30: segmented_labels = pixel_labels.reshape(rows, cols)
31:
32: # 5. Buat citra tersegmentasi (warnai setiap segmen dengan warna rata-
    rata klaster)
33: segmented_image_kmeans = np.zeros_like(image_lab)
34: centers_lab = kmeans.cluster_centers_
35: for k in range(n_clusters):
36:     # Dapatkan piksel yang termasuk klaster k
37:     cluster_pixels = (pixel_labels == k)
38:     # Isi piksel tersebut di citra baru dengan warna pusat klaster k
39:     # Perlu reshape kembali cluster_pixels ke bentuk citra
40:     mask_k = cluster_pixels.reshape(rows, cols)
41:     segmented_image_kmeans[mask_k] = centers_lab[k]
42:
43: # Konversi kembali ke RGB untuk ditampilkan
44: segmented_image_rgb = lab2rgb(segmented_image_kmeans)
45:
46: # 6. Visualisasi Hasil
47: fig, axes = plt.subplots(ncols=3, figsize=(12, 4))
48: ax = axes.ravel()
49:
50: ax[0].imshow(image_astro)
51: ax[0].set_title('Original Image (RGB)')
52: ax[0].axis('off')
53:
54: # Menampilkan label klaster
55: ax[1].imshow(segmented_labels, cmap='viridis') # cmap bisa diganti
56: ax[1].set_title(f'K-Means Labels (K={n_clusters})')
57: ax[1].axis('off')
58:
59: # Menampilkan citra hasil segmentasi K-Means
60: ax[2].imshow(segmented_image_rgb)
61: ax[2].set_title(f'Segmented Image (K-Means, K={n_clusters})')
62: ax[2].axis('off')
63:
64: plt.tight_layout()
65: plt.show()
```


Hasil Praktikum:



Amati hasil segmentasi K-Means. Apakah jumlah kluster (K) mempengaruhi hasil segmentasi secara signifikan? Cobalah mengubah nilai K. Apakah penggunaan ruang warna Lab membantu?

Praktikum 4. Segmentasi Berbasis Tepi Menggunakan Watershed

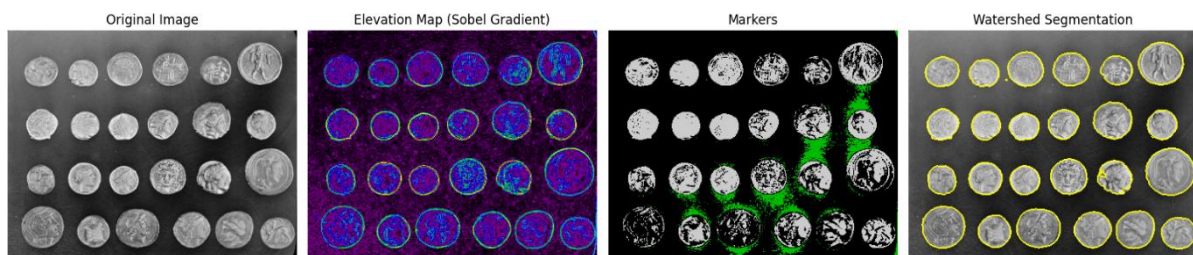
Praktikum ini memperkenalkan algoritma Watershed, yang sering digunakan untuk memisahkan objek yang saling bersentuhan. Algoritma ini memperlakukan citra seperti relief topografi dan mengisi 'cekungan' (basin) dengan air. Kita akan menggunakan gradien citra sebagai input topografi.

Kode Praktikum:

```
01: import numpy as np
02: import matplotlib.pyplot as plt
03: from skimage import data, filters, segmentation, morphology, measure
04: from scipy import ndimage as ndi
05:
06: # 1. Memuat citra (contoh: coins)
07: image_coins = data.coins()
08:
09: # 2. Hitung gradien citra (sebagai 'topografi')
10: elevation_map = filters.sobel(image_coins)
11:
12: # 3. Tentukan marker (penanda awal untuk setiap cekungan/objek)
13: # Kita bisa menggunakan thresholding untuk mendapatkan marker kasar
14: markers = np.zeros_like(image_coins)
15: markers[image_coins < 30] = 1 # Marker untuk latar belakang
16: markers[image_coins > 150] = 2 # Marker untuk objek (koin)
17:
18: # Atau gunakan marker dari local maxima (lebih canggih)
19: # from skimage.feature import peak_local_max
20: # image_max = ndi.maximum_filter(image_coins, size=20, mode='constant')
21: # coordinates = peak_local_max(image_coins, min_distance=20)
22: # markers = np.zeros(image_coins.shape, dtype=bool)
23: # markers[tuple(coordinates.T)] = True
24: # markers = measure.label(markers)
25:
26: # 4. Terapkan algoritma Watershed
27: segmentation_watershed = segmentation.watershed(elevation_map, markers)
28:
29: # Warnai hasil segmentasi untuk visualisasi
30: segmented_colored = segmentation.mark_boundaries(image_coins,
segmentation_watershed)
31:
32: # 5. Visualisasi Hasil
33: fig, axes = plt.subplots(ncols=4, figsize=(16, 4), sharex=True,
sharey=True)
34: ax = axes.ravel()
35:
36: ax[0].imshow(image_coins, cmap=plt.cm.gray)
37: ax[0].set_title('Original Image')
38: ax[0].axis('off')
39:
40: ax[1].imshow(elevation_map, cmap=plt.cm.nipy_spectral)
41: ax[1].set_title('Elevation Map (Sobel Gradient)')
42: ax[1].axis('off')
```

```
43:
44: ax[2].imshow(markers, cmap=plt.cm.nipy_spectral)
45: ax[2].set_title('Markers')
46: ax[2].axis('off')
47:
48: ax[3].imshow(segmented_colored)
49: ax[3].set_title('Watershed Segmentation')
50: ax[3].axis('off')
51:
52: plt.tight_layout()
53: plt.show()
```

Hasil Praktik



Amati bagaimana algoritma Watershed menggunakan gradien dan marker untuk memisahkan objek. Bagaimana pemilihan marker mempengaruhi hasil akhir? Apa yang terjadi jika marker tidak tepat?

Praktikum 5. Perbandingan Visual Hasil Segmentasi

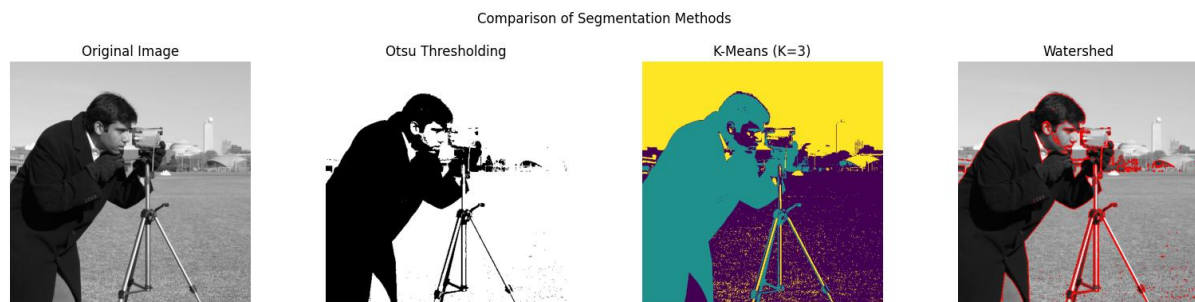
Praktikum ini bertujuan membandingkan hasil dari beberapa metode segmentasi yang telah dipelajari pada citra yang sama untuk melihat perbedaan, kelebihan, dan kekurangan masing-masing secara visual.

Kode Praktikum:

```
01: import matplotlib.pyplot as plt
02: from skimage import data, filters, segmentation, img_as_float, color
03: from sklearn.cluster import KMeans
04: import numpy as np
05: import warnings
06:
07: # 1. Pilih satu citra untuk perbandingan (misal: camera)
08: image = data.camera()
09: image_float = img_as_float(image)
10:
11: # 2. Lakukan beberapa metode segmentasi
12: # a) Otsu Thresholding
13: thresh_otsu = filters.threshold_otsu(image)
14: binary_otsu = image > thresh_otsu
15:
16: # b) K-Means (misal K=3)
17: # Reshape untuk K-Means (1 fitur: intensitas)
18: rows, cols = image.shape
```

```
19: pixel_features = image_float.reshape(rows * cols, 1)
20: n_clusters = 3
21: kmeans = KMeans(n_clusters=n_clusters, random_state=0, n_init=10)
22: with warnings.catch_warnings():
23:     warnings.simplefilter("ignore")
24:     pixel_labels = kmeans.fit_predict(pixel_features)
25: segmented_kmeans_labels = pixel_labels.reshape(rows, cols)
26:
27: # c) Watershed (gunakan marker sederhana dari Otsu)
28: elevation_map = filters.sobel(image)
29: markers = np.zeros_like(image)
30: markers[image < thresh_otsu] = 1
31: markers[image > thresh_otsu] = 2
32: segmentation_watershed = segmentation.watershed(elevation_map, markers)
33:
34: # 3. Visualisasi Perbandingan
35: fig, axes = plt.subplots(1, 4, figsize=(16, 4), sharex=True, sharey=True)
36: ax = axes.ravel()
37:
38: ax[0].imshow(image, cmap=plt.cm.gray)
39: ax[0].set_title('Original Image')
40: ax[0].axis('off')
41:
42: ax[1].imshow(binary_otsu, cmap=plt.cm.gray)
43: ax[1].set_title('Otsu Thresholding')
44: ax[1].axis('off')
45:
46: ax[2].imshow(segmented_kmeans_labels, cmap='viridis') # Gunakan cmap
berbeda untuk label
47: ax[2].set_title(f'K-Means (K={n_clusters})')
48: ax[2].axis('off')
49:
50: # Gunakan mark_boundaries untuk Watershed agar lebih jelas
51: segmented_watershed_colored = segmentation.mark_boundaries(image_float,
segmentation_watershed, color=(1,0,0)) # Batas merah
52: ax[3].imshow(segmented_watershed_colored)
53: ax[3].set_title('Watershed')
54: ax[3].axis('off')
55:
56: plt.suptitle('Comparison of Segmentation Methods')
57: plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Adjust layout to make space
for suprtile
58: plt.show()
```

Hasil Praktik



Bandingkan hasil ketiga metode pada citra yang sama. Metode mana yang paling baik dalam memisahkan objek utama? Metode mana yang menghasilkan region paling halus atau paling detail? Apa kelebihan dan kekurangan masing-masing metode berdasarkan hasil visual ini?

E. Hasil Praktik

Lengkapi tabel hasil praktikum berikut:

No.	Nama Praktikum	Hasil Praktikum
1	Segmentasi Menggunakan Thresholding Global dan Otsu	
2	Segmentasi Menggunakan Region Growing (Contoh Sederhana)	
3	Segmentasi Citra Berwarna Menggunakan K-Means Clustering	
4	Segmentasi Berbasis Tepi Menggunakan Watershed	
5	Perbandingan Visual Hasil Segmentasi	

F. Penugasan

1. Kumpulkan Laporan Praktikum dari jobsheet ini dalam bentuk Microsoft word sesuai dengan format jobsheet praktikum dan dikumpulkan di web elnino polines. (JANGAN DALAM BENTUK PDF)
2. Kumpulkan luaran kode praktikum dalam bentuk ipynb yang sudah diunggah pada akun github masing-masing. Lampirkan tautan github yang sudah di unggah melalui laman LMS elnino.
3. **Eksperimen Thresholding:** Terapkan metode thresholding Otsu pada citra `data.page()`. Apakah hasilnya memuaskan? Mengapa? Coba gunakan metode thresholding lain yang ada di `skimage.filters` (misalnya, `threshold_local` atau `threshold_yen`) dan bandingkan hasilnya.
4. Cari satu contoh citra sederhana dari internet (misalnya, citra buah-buahan dengan latar belakang polos, atau citra sel mikroskop). Terapkan setidaknya tiga metode segmentasi yang berbeda (misalnya, Otsu, K-Means, dan Watershed) pada citra tersebut. Bandingkan hasilnya secara visual dan diskusikan metode mana yang paling cocok untuk citra pilihan Anda, serta jelaskan alasannya

G. Kesimpulan

Segmentasi citra adalah proses fundamental dalam pengolahan citra yang membagi citra menjadi region-region bermakna berdasarkan kesamaan properti piksel. Berbagai metode segmentasi tersedia, mulai dari thresholding yang sederhana, metode berbasis region seperti region growing dan watershed, metode berbasis kontur seperti active contours, hingga metode clustering seperti K-Means. Setiap metode memiliki karakteristik, kelebihan, dan keterbatasan:

- **Thresholding:** Cepat, efektif untuk kontras baik, namun sensitif terhadap noise dan pencahayaan.
- **Region Growing:** Intuitif, menghasilkan region terhubung, sensitif pada seed dan kriteria.
- **K-Means:** Dapat mengelompokkan berdasarkan fitur, perlu menentukan K, sensitif pada inisialisasi.
- **Watershed:** Baik untuk memisahkan objek bersentuhan, sensitif terhadap noise (memerlukan marker/pra-pemrosesan).
- **Active Contours:** Baik untuk objek dengan batas jelas, memerlukan inisialisasi kontur yang baik, bisa jadi lambat secara komputasi dan sensitif terhadap parameter.

Pemilihan metode segmentasi yang tepat sangat bergantung pada karakteristik citra (misalnya, kontras, noise, kompleksitas objek) dan tujuan akhir aplikasi. Seringkali, pra-pemrosesan atau kombinasi beberapa teknik diperlukan untuk hasil optimal.

H. Referensi

Munir, Rinaldi. Pengolahan Citra Digital – Bab 7: Perbaikan Kualitas Citra, Institut Teknologi Bandung. (Menjelaskan definisi peningkatan kualitas citra secara matematis dan konsep-konsep dasar)

Faradilla, Putri, et al. (2022). “Implementasi Metode Kernel Konvolusi dan Contrast Stretching untuk Perbaikan Kualitas Citra Digital.” Jurnal Sistem Informasi TGD, Vol. 1 No. 6, 865-874. (Sumber konsep point processing dan konvolusi pada domain spasial)

Wikipedia (Bahasa Indonesia). “Perataan histogram.” (Menjelaskan konsep histogram citra dan teknik ekualisasi histogram sebagai metode peningkatan kontras)

Wikipedia (Bahasa Indonesia). “Filter median.” (Menjelaskan prinsip filter median sebagai penyaring non-linear untuk menghilangkan derau impuls, serta keunggulannya dalam menjaga tepi citra)

.